Institut National des Postes et Télécommunications (INPT)

Devoir Mathématique

Solving Linear Systems: Comparison of Pivoting Strategies in Gaussian Elimination

Submitted by: **EL MEHDAOUI Chaimaa**

Academic year 2026/2027

Source code: `github.com/chximaa/Total-Pivoting`

Introduction

Solving linear systems $Ax = b$ is one of the central problems of numerical analysis. Gaussian elimination, which reduces the system to triangular form, is the most widely used direct method. However, when a pivot is zero or very small, the algorithm becomes unstable. To avoid this, pivoting strategies are used.

This report compares three variants of Gaussian elimination on random matrices, in terms of CPU time and solution accuracy: row swapping, partial pivoting and total pivoting.

1 Theoretical background

1.1 Gaussian elimination

Let $A \in \mathbb{R}^{n\times n}$ be invertible and $b \in \mathbb{R}^n$. For $k = 1, \dots, n-1$, compute the multipliers

$$m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \qquad i = k+1, \dots, n,$$

then update the matrix:

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik}\, a_{kj}^{(k)}, \qquad b_i^{(k+1)} = b_i^{(k)} - m_{ik}\, b_k^{(k)}.$$

The resulting system is upper triangular and is solved by back substitution. The number of elementary operations is of order $\frac{2}{3}n^3$, so the complexity of the method is $\mathcal{O}(n^3)$.

1.2 Pivoting strategies

- **Row swapping (zero pivot).** Two rows are swapped only when the pivot $a_{kk}^{(k)}$ is zero. This is the simplest case, but it does not protect against very small pivots.
- **Partial pivoting.** At step k, choose the row $p \geq k$ such that $|a_{pk}^{(k)}| = \max_{i\geq k} |a_{ik}^{(k)}|$, then swap rows k and p. This yields a factorization $PA = LU$.
- **Total pivoting.** At step k, search for the element of largest modulus in the whole submatrix $(a_{ij}^{(k)})_{i,j\geq k}$, then swap rows and columns. This yields $PAQ = LU$. The unknowns are then permuted, and their original order must be restored at the end.

The cost of the pivot search differs between strategies: $\mathcal{O}(n-k)$ comparisons for partial pivoting, and $\mathcal{O}((n-k)^2)$ for total pivoting. Summed over all steps, total pivoting requires about $\frac{n^3}{3}$ comparisons, which is of the same order as the elimination itself.

2 Experimental protocol

The experiments were carried out in Python, using NumPy for vectorized operations on matrix rows. The complete Python code is available on GitHub: `https://github.com/chximaa/Total-`
For each size n:

1. Generate a matrix A whose entries follow the uniform distribution on $[-1, 1]$, with a fixed seed for reproducibility.
2. Choose a random exact solution x_{ex} and set $b = A\,x_{\text{ex}}$.
3. Solve $Ax = b$ with each of the three methods and measure the CPU time with `time.process_time()`.
4. Compute the relative error $\dfrac{\|x - x_{\text{ex}}\|_2}{\|x_{\text{ex}}\|_2}$ and the relative residual $\dfrac{\|Ax - b\|_2}{\|b\|_2}$.

For $n = 20000$, a dense double-precision matrix occupies about 3.2 GB and the elimination requires about $5.3 \cdot 10^{12}$ operations. The real running time could therefore not be measured directly. It is estimated from the model $T(n) = c\,n^3$, where c is calibrated on the largest measured size ($n = 1000$). Since this is an extrapolation by a factor of 20 in n, the figures should be read as orders of magnitude only, and optimized compiled libraries would be far faster than this Python implementation.

3 Results

n	Method	CPU (s)	Relative error	Relative residual
100	Row swapping	0.0024	9.24e-13	1.27e-13
100	Partial pivoting	0.0032	1.07e-14	1.42e-15
100	Total pivoting	0.0043	1.08e-14	1.39e-15
200	Row swapping	0.0111	1.48e-12	2.19e-13
200	Partial pivoting	0.0091	2.61e-14	2.74e-15
200	Total pivoting	0.0169	2.99e-14	1.57e-15
400	Row swapping	0.0606	2.29e-10	4.14e-13
400	Partial pivoting	0.0582	1.45e-11	4.72e-15
400	Total pivoting	0.0912	2.42e-12	3.44e-15
600	Row swapping	0.1887	5.93e-11	4.24e-12
600	Partial pivoting	0.1794	1.94e-13	7.09e-15
600	Total pivoting	0.2910	8.10e-14	4.11e-15
800	Row swapping	0.3833	1.03e-11	1.58e-12
800	Partial pivoting	0.4629	1.14e-13	8.71e-15
800	Total pivoting	0.7505	1.07e-13	5.63e-15
1000	Row swapping	0.7965	7.15e-11	8.12e-12
1000	Partial pivoting	0.8845	3.90e-13	1.18e-14
1000	Total pivoting	1.2178	1.14e-13	6.09e-15

Table 1: Experimental results (CPU time and accuracy)

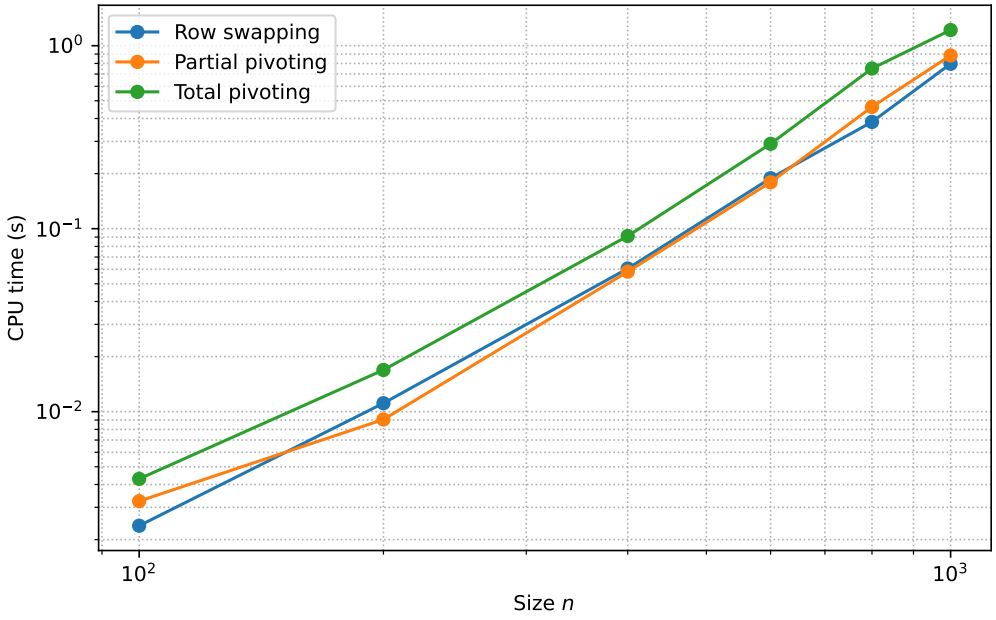


Figure 1: CPU time as a function of size n (log-log scale)

Method	Estimated time for $n = 20000$ (s)	Estimated time (h)
Row swapping	6.372e+03	1.77
Partial pivoting	7.076e+03	1.97
Total pivoting	9.743e+03	2.71

Table 2: Extrapolated CPU time for $n = 20000$ (model $T(n) = c\,n^3$)

4 Analysis and discussion

In terms of computation time, all three methods follow the same cubic law, so the curves are roughly parallel on a log-log plot. Row swapping and partial pivoting have very similar running times, because the linear pivot search of partial pivoting is negligible compared with the $\mathcal{O}(n^2)$ update at each step. Total pivoting is consistently the slowest, since it searches the whole submatrix and permutes columns, which costs roughly 40 to 60% more than the other two in these experiments.

In terms of accuracy, the difference is clear. Row swapping, which does not protect against small pivots, gives the largest errors (up to about 10^{-10} for $n = 400$), while partial and total pivoting stay close to machine precision. Total pivoting is generally the most accurate, but its gain over partial pivoting is small on random matrices. Partial pivoting therefore offers the best compromise between cost and stability, which is why it is used in reference libraries such as LAPACK.

For $n = 20000$, the estimated times show that direct solution with this implementation would be very expensive. In practice, compiled and parallel libraries, or iterative methods suited to sparse matrices, are used instead.

Conclusion

This study compares three variants of Gaussian elimination on random matrices. Row swapping alone is the least accurate. Total pivoting is the most expensive in computation time for only a small gain in accuracy. Partial pivoting offers the best compromise between speed and numerical stability.